



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 1

Project Title: Linux System Information and Resource Monitoring Tool

Team Members

Roll Number	Student Name
2520030554	Sai Hima Sekhar
2520030019	V. Navadeep
2520030063	K. Shankar

1. Problem Statement

Linux systems manage various resources such as CPU, memory, processes, and other system components continuously. However, obtaining this information in a simple and organized manner can be difficult for users without directly interacting with Linux system interfaces and commands. There is a need for a lightweight user-space tool that can collect and display important system information and resource usage in a clear format.

The proposed project addresses this problem by developing a Linux System Information and Resource Monitoring Tool using C/C++ and Linux/POSIX system-programming mechanisms. The tool will access Linux system information, particularly through the /proc filesystem and file I/O mechanisms, to obtain details about processes, CPU utilization, memory usage, and overall system status. The project is directly relevant to Operating Systems and Systems Programming because it demonstrates how user-space applications interact with Linux system interfaces and monitor resources managed by the operating system.

2. Objectives

- To develop a Linux-based system information and resource monitoring tool using C/C++.
- To collect and display important system information such as CPU, memory, processes, and system uptime.

- To monitor system resources such as CPU and memory utilization and present the information in a clear format.
- To demonstrate Operating Systems and Systems Programming concepts through Linux system interfaces, file handling, system calls, and process management.

3. Selected Commands and Their Workflow

The project is implemented as a **modular C application** rather than a collection of independent shell commands. The major workflow can therefore be understood through the Linux interfaces and program functions used by each module.

→ System Information Workflow

The first stage collects general information about the Linux system. The program uses `gethostname()` to obtain the hostname, reads `/proc/version` to obtain kernel information, uses `uname()` to determine the system architecture, and reads `/proc/uptime` to obtain the system uptime. These values are stored and then displayed in the System Information section of the application.

→ Memory Monitoring Workflow

The memory-monitoring stage reads `/proc/meminfo` and extracts values such as `MemTotal`, `MemAvailable`, and `MemFree`. The program calculates used memory by subtracting available memory from total memory. It then calculates the memory utilization percentage and displays the total, available, free, used memory, and usage percentage to the user.

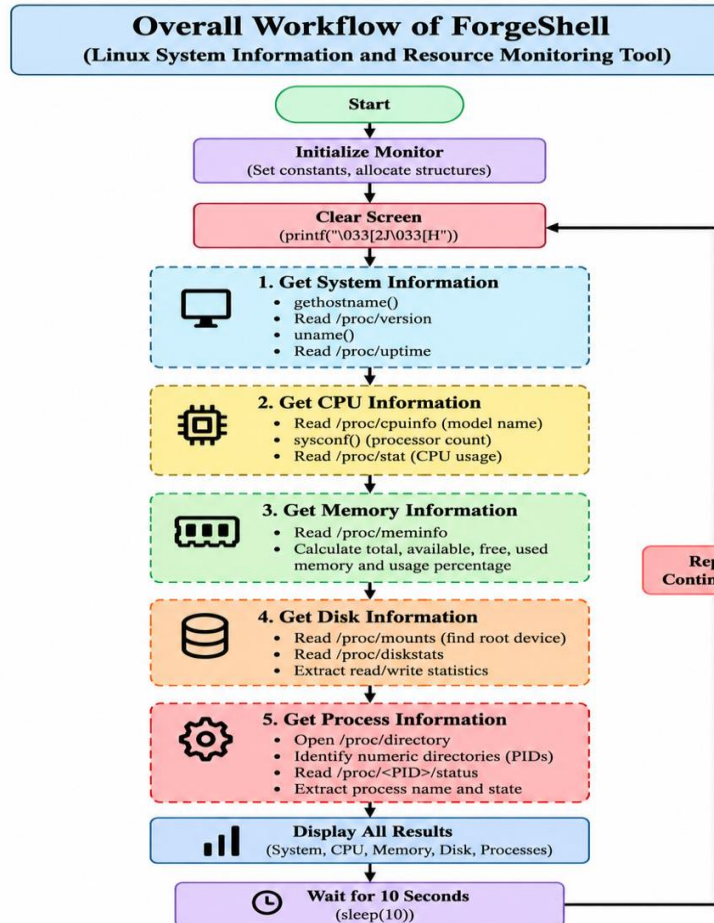
→ CPU Information

The next stage monitors the processor. The program reads `/proc/cpuinfo` to obtain the CPU model name and uses `sysconf(_SC_NPROCESSORS_ONLN)` to determine the number of online logical processors. For CPU utilization, it reads `/proc/stat` twice with a one-second interval between the readings. The difference between the two readings is used to calculate the current CPU usage percentage.

→ Disk Information

The disk-monitoring stage first reads `/proc/mounts` to identify the device associated with the root filesystem `/`. After identifying the device, the program reads `/proc/diskstats` and searches for the corresponding device entry. It extracts the number of completed reads and writes, sectors read and written, and converts the sector values into megabytes for display.

→ Overall Workflow



4. Implementation

The **Linux System Information and Resource Monitoring Tool** is implemented as a modular, user-space C application for Linux. The implementation is divided into separate modules for system information, CPU monitoring, memory monitoring, disk monitoring, process monitoring, and common file-handling operations. This modular design makes the program easier to understand, test, and maintain. The application obtains system-level information primarily through the Linux `/proc` filesystem and standard Linux/POSIX system calls and APIs, which directly supports the Operating Systems concepts identified in the project abstract

5. Detailed Description of the Implementation

The project is implemented using modular C programs to monitor different Linux system resources. The `system_info` module collects the hostname, kernel version, system architecture, and uptime using Linux system interfaces. The `cpu_monitor` module reads CPU statistics from `/proc/stat` and calculates CPU utilization using two snapshots taken one second apart. The `memory_monitor` module reads `/proc/meminfo` to calculate total, available, free, and used memory. The `disk_monitor` module identifies the root filesystem through `/proc/mounts` and obtains disk read/write statistics from `/proc/diskstats`. The `process_monitor` module scans the

/proc directory and retrieves process IDs, names, and states from process status files. A common utility function handles file reading using open(), read(), and close(). Finally, main.c integrates all modules and refreshes the displayed information every 10 seconds.

→ System Information Module

The module retrieves basic system details such as hostname, kernel version, architecture, and uptime using gethostname(), uname(), and Linux /proc files.

→ CPU Monitoring Module

CPU information is obtained from /proc/cpuinfo and /proc/stat. CPU usage is calculated by comparing total and idle CPU time between two measurements separated by one second.

→ Memory Monitoring Module

The memory module reads /proc/meminfo and extracts MemTotal, MemAvailable, and MemFree. It then calculates used memory and memory usage percentage.

→ Disk Monitoring Module

The disk module detects the device mounted at / and reads its statistics from /proc/diskstats. It reports completed reads/writes, sectors, and data transferred in MB.

→ Process Monitoring Module

The process module scans /proc, identifies numeric directories as process IDs, and reads process names and states from /proc/[PID]/status

6. Test Cases and Results

Start the monitoring program

```
navadeep@LAPTOP-62I4PSPU:~/Kernel_Kings_OSSP/ForgeOS/Project$ make run
./forgeos_monitor
=====
LINUX SYSTEM INFORMATION & RESOURCE MONITOR
=====

SYSTEM INFORMATION
-----
Hostname       : LAPTOP-62I4PSPU
Kernel Version : Linux version 6.18.33.2-microsoft-standard-WSL2 (root@f1bbfb02316b) (gcc (GCC) 13.2.0, GNU ld (GNU Binutils) 2.41) #1 SMP PREEMPT_DYNAMIC Thu Jun 18 2
1:54:43 UTC 2026
Architecture   : x86_64
Uptime         : 216.51 seconds
```

Check CPU monitoring

```
CPU INFORMATION
-----
Model          : Intel(R) Core(TM) i3-7100U CPU @ 2.40GHz
Processors     : 4
CPU Usage      : 0.00%
```

Check memory monitoring

```
MEMORY INFORMATION
-----
Total Memory      : 3943024 kB
Available Memory  : 3482744 kB
Free Memory       : 2242852 kB
Used Memory       : 460280 kB
Memory Usage      : 11.67%
```

Check disk monitoring

```
DISK INFORMATION
-----
Device           : sdd
Reads Completed  : 3968
Sectors Read     : 1309042
Data Read        : 639.18 MB
Writes Completed : 692
Sectors Written  : 39272
Data Written     : 19.18 MB
```

Check process monitoring

PROCESS MONITOR		
PID	NAME	STATE
1	systemd	S
2	init-systemd(Ub	S
6	init	S
57	systemd-journal	S
106	systemd-udevd	S
164	systemd-resolve	S
165	systemd-timesyn	S
180	cron	S
181	dbus-daemon	S
194	systemd-logind	S
199	wsl-pro-service	S
211	rsyslogd	S
232	agetty	S
251	unattended-upgr	S
417	SessionLeader	S
418	Relay(419)	S
419	bash	S
420	login	S
485	systemd	S
486	(sd-pam)	S
520	bash	S
659	make	S
660	forgeos_monitor	R
Total processes displayed: 23		

Wait for 10 seconds

```

=====
Refreshing every 10 seconds | Press Ctrl+C to exit
=====
LINUX SYSTEM INFORMATION & RESOURCE MONITOR
=====

SYSTEM INFORMATION
-----
Hostname       : LAPTOP-62I4PSPU
Kernel Version : Linux version 6.18.33.2-microsoft-standard-WSL2 (root@f1bbfb02316b) (gcc (GCC) 13.2.0, GNU ld (GNU Binutils) 2.41) #1 SMP PREEMPT_DYNAMIC Thu Jun 18 2
1:54:43 UTC 2026
Architecture   : x86_64
Uptime         : 227.51 seconds

CPU INFORMATION
-----
Model          : Intel(R) Core(TM) i3-7100U CPU @ 2.40GHz
Processors     : 4
CPU Usage      : 0.00%

MEMORY INFORMATION
-----
Total Memory   : 3943024 kB
Available Memory : 3482744 kB
Free Memory    : 2242852 kB
Used Memory    : 460280 kB
Memory Usage   : 11.67%

```

Press Ctrl+C to exit

```

=====
Refreshing every 10 seconds | Press Ctrl+C to exit
=====
^Cmake: *** [Makefile:30: run] Interrupt

```

7. Conclusion

The **Linux System Information and Resource Monitoring Tool** was successfully implemented in C as a user-space Linux application. The project demonstrates practical Operating Systems and Systems Programming concepts through the /proc filesystem, Linux file I/O, system calls, process monitoring, and resource management. The tool successfully collects and displays CPU, memory, disk, process, and system information through a command-line interface. The periodic refresh mechanism allows users to observe changes in system resources over time. Overall, the project provides practical experience in interacting with Linux system interfaces and understanding how user-space programs obtain information managed by the operating system.